

Assignment 2

INF-2300

Erling Heimstad Willassen

1. Introduction

Clients and servers can communicate with each other using CRUD (Create, Read, Update, Delete) operations through HTTP (HyperText Transfer Protocol). This form of communication allows a client to receive a web page layout written in HTML (HyperText Markup Language) and JavaScript.

In this assignment, a web page in HTML was created to communicate with a provided server implemented in JavaScript. The server displays and allows manipulation of a list stored on the server side.

2. Technical Background

HTML is a very much used markup language to display web pages on the internet. HTML is a document file that contains code describing the layout of the page content and how it will be displayed by a client browser [1]. By entering a domain name or IP address into a browser [2], the browser sends an HTTP request to the corresponding server to retrieve the HTML layout, files, or other resources.

The server handles these requests according to predefined rules, allowing or denying access to certain resources and returning data to the client. In this way, the server determines what the client can access and display through HTTP requests. Furthermore, these HTTP requests can also be used to manipulate the internal state of objects or files on the server if the server provides appropriate handling methods.

JavaScript is a lightweight programming language commonly used on web pages to provide interactivity and dynamic functionality [3]. Web browsers have built-in JavaScript engines that can execute JavaScript code embedded in HTML documents [3]. While HTML alone provides a static layout, JavaScript allows dynamic interaction by executing functions tied to HTML elements. This integration enables communication with a server to fetch or modify information displayed on the client side.

Flask is a web application framework for the Python programming language that simplifies the process of creating and running web servers [4]. Flask includes built-in methods for handling client requests and responses, making it easy to develop a working server in a short amount of time.

3. Design

The goal of the implementation was to create a static HTML web page displayed on the client side. This file would be hosted on the server and accessible to clients by sending a "GET" request to either "/" or "/index.html". The HTML file defined the layout of the web page, containing elements that triggered different HTTP requests.

These elements were connected to JavaScript functions within the same HTML file, where the HTTP requests were handled and displayed in an organized manner. Since the interface needed to be user-friendly, each interactive object was labeled descriptively and included error handling to minimize invalid requests.

The provided server code already included predefined request routes. These routes were used to modify a list stored on the server side. By interacting with objects on the client page, users could manipulate the state of this list through HTTP requests.

4. Implementation

As described in Section 3, the assignment included a pre-written Python server using Flask. This code already supported various client requests, except for serving the index.html file. To fix this, a route was added so that when a client accessed the server's IP or domain (or appended /index.html), the server would return the HTML layout using Flask's `render_template()` function.

```
@app.route("/", methods=["GET"])
def index():
    return render_template('index.html')

@app.route("/index.html", methods=["GET"])
def index_html():
    return render_template('index.html')
```

Figure 1: Code for routing a GET request to client

The HTML file was created from scratch. Using a basic HTML template, the web page displayed input fields and buttons that interacted with the server to manipulate a list object named `items`, which contained a dictionary/JSON of TODO entries stored on the server.

```
items = [
    {"id": 1, "name": "Create a client application", "done": False},
    {"id": 2, "name": "Write an excellent report", "done": False},
    {"id": 3, "name": "Jazz it up with some cool features?", "done": False},
]
```

Figure 2: Item list on server side to have its state altered by HTTP requests.

One of the main challenges was connecting HTML elements to JavaScript functions. This was solved by wrapping input fields in a `<form></form>` container and using the `onsubmit=` event listener to trigger specific JavaScript functions.

The `fetch()` method in JavaScript was then used to send HTTP requests to the server. Each CRUD operation was implemented as a separate JavaScript function that interacted with the items list depending on user actions in the HTML interface.

To improve responsiveness, asynchronous functions were used with the `async` keyword. Normally, JavaScript functions execute synchronously, which means other code pauses until a task is complete. By using `async`, functions can run concurrently, preventing the web page from freezing while waiting for a server response.

The `await` keyword was combined with `fetch()` to pause function execution until the server response was received, preventing errors caused by trying to access data before it arrived. Additionally, `e.preventDefault()` was used to stop the browser from reloading the page automatically, allowing the web page to update dynamically without refreshing.

```
async function submitID(e){ // Get Query / Submit ID GET function
  e.preventDefault();
  var numberID = document.getElementById("inputID").value.trim();
  const response = await fetch(`/api/items/${encodeURIComponent(numberID)}`);
  const object = await response.json();
  const status = document.getElementById('getStatus');
  if (!object.item){
    status.innerHTML = object.Description;
    return false
  }
  status.innerHTML = ""; // resetting text in case of earlier error code
  createTable(object)
}
```

Figure 3: An example of how a JavaScript function looks like in the HTML file.

5. Results

When the server's IP address was accessed from a client browser as "127.0.0.1:8080" or "localhost:8080", the "index.html" layout was requested and correctly displayed as seen below in figure 4.

TODO List

Instructions:
This site will handle your TODO list.
You will get 7 options to use on your TODO list

- 1. Submit Text Entry to List:**
Sends text entry to server that will append to "TODO" list.
- 2. Get Text Entry from List:**
Request to get specific text entry from "TODO" list with ID number.
- 3. Change Text Entry in List:**
Changes a text entry in "TODO" list with ID number.
- 4. Delete Text Entry in List:**
Deletes a text entry in "TODO" list with ID number.
- 5. Click to Get List:**
Displays all entries of text in "TODO" list.
- 6. Click to Remove List:**
Removes all entries in the "TODO" list.
- 7. Click on "Done"**
Changes a text entry to "Done" to represent completion of activity

Submit Text
New text to put in list
Submit text entry to insert into TODO list.

Get Text
Enter ID number
Submit Text Entry ID from list to be displayed.

Change Text
Enter ID number New text for Entry ID
Submit Text Entry ID in list to change text.

Delete Text
Enter ID number
Submit Text Entry ID in list to be deleted

TODO list

ID	Name	Done
1	Create a client application	☐
2	Write an excellent report	☐
3	Jazz it up with some cool features?	☐

Figure 4: How the HTML layout looks in a web-client.

The user could perform seven actions through the web interface. Each action was tied to a specific JavaScript function and HTTP request:

Submit Query	Sends text entry to server that will append to "TODO" list
Get Query	Request to get specific text entry from "TODO" list with ID number.
Change Query	Changes a text entry in "TODO" list with ID number.
Delete Query	Deletes a text entry in "TODO" list with ID number.
Click to fetch list	Displays all entries of text in "TODO" list.
Click to remove list	Removes all entries in the "TODO" list.
Click "Done"	Changes a text entry to "Done" to represent completion of activity

All actions worked as intended and successfully modified the server-side list.

6. Discussion

The static HTML web page provided an efficient way to display and manipulate the server-side list. This approach was chosen because it offered a simple and straightforward method to create a working client interface without the need for a complex front-end framework.

Developing the connection between JavaScript and the server took some effort. Initially, errors occurred when the client attempted to process responses before the server had finished sending them. This issue was solved by implementing asynchronous functions with the `await` keyword, ensuring that the JavaScript waited for server responses before proceeding. Once this was implemented, the overall functionality became much more stable and predictable.

Although only minor changes were made to the Python server, using Flask proved to be an efficient and user-friendly way to set up a server. This demonstrates that creating a functional web server can be achieved quickly and with relatively little code. The flask module handles most of the technical back-end code of a server, while the programmer has to handle what the responses should be to each request.

7. Conclusion

A web client with a user-friendly interface was successfully implemented to handle HTTP requests for manipulating a server-side list. The static HTML web page, enhanced with JavaScript functionality, allowed users to view, add, modify, and delete list items on the server. The project demonstrated effective client-server communication using Flask, HTML, and JavaScript.

8. Sources

- [1] Wikipedia Contributors, "HTML," *Wikipedia*, Nov. 27, 2018.
<https://en.wikipedia.org/wiki/HTML>
- [2] Wikipedia Contributors, "Domain Name System," *Wikipedia*, Aug. 09, 2019.
https://en.wikipedia.org/wiki/Domain_Name_System
- [3] Wikipedia Contributors, "JavaScript," *Wikipedia*, Mar. 22, 2019.
<https://en.wikipedia.org/wiki/JavaScript>
- [4] Flask, "Welcome to Flask — Flask Documentation (3.0.x)," *Palletsprojects.com*, 2010.
<https://flask.palletsprojects.com/en/stable/>